TermiNuke Battle Arena

CSC209 Assignment 3 — Systems Programming Mini-Project

Project Documentation

Contents

Project Overview

TermiNuke is a multiplayer, turn-based, interactive battle arena played entirely inside the terminal. It falls under Category 2: Client-Server Application Using Sockets. Players connect to a server that hosts up to 64 independent game rooms. Each client first selects a room from a live room-list screen, then joins the lobby of that room. Once in the lobby, players select one of eight unique characters and battle against opponents using the character's available actions. The server records and broadcasts the status of the player and the battle process. When one player's HP reaches 0 or quits the game, the other player is declared the winner by the server.

What the application does

The server tracks all game states, validates every player action, and resolves combat, which means that it handles basically all aspects of the game. Clients are basically just terminals that send player input and displays text through server broadcasts.

The game proceeds through phases that form a state machine (declared in `game.h`, lines 12–17):

1. **Room Selection**: Immediately after TCP connection, the server sends `MSG_ROOM_LIST` to the new client. The client displays a table of all available rooms with their player count, status (Lobby / In Progress), and spectator count. The player types `join <room_id>` to enter a specific room or `new` to be auto-assigned to an empty room.
2. **Lobby**: Players connect to a room and select one of eight characters from a displayed roster indexed 0–7. Each character can only be picked once, which will be displayed as "TAKEN". The first player to join becomes the host.
3. **Action Collection**: The host starts the game. Each round, every living player submits one action within a 30-second countdown timer. Some actions require a target to be an input during submission as well, which is indicated in the action description. The server accepts submissions at any time during this window and confirms each with a `MSG_WAIT` acknowledgement.
4. **Resolution**: When all actions are in or the timer expires (whichever comes first), the server executes moves in speed order. Speed is preset for each character and can be impacted with some actions. The moves to be executed are put in a queue of up to 16 narrative event strings, and they are released to every client at two-second intervals for dramatic effect.
5. **Game Over**: The server announces the winner (or a tie if all remaining players are eliminated simultaneously) via `MSG_GAME_OVER`. The room then automatically resets to the lobby phase, and any spectators who were watching are promoted to lobby players for the next game.

Spectator mode

If a client selects a room that has a game already in progress, they join as a *spectator* instead of a player. The server sends `MSG_SPECTATE_START` with a snapshot of the current game state. Spectators receive all subsequent game broadcasts (`MSG_ROUND_START`, `MSG_TIMER_TICK`, `MSG_ROUND_EVENT`, `MSG_PLAYER_ELIM`, `MSG_GAME_OVER`) exactly as players do. When the game ends, `promote_spectators` (server.c, line 409) automatically transfers each spectator's file descriptor into an empty player slot and sends them `MSG_JOIN_ACK` and `MSG_CHAR_LIST`, seamlessly placing them in the lobby for the next round.

Input and output

Server: No command line arguments are required. It listens on the port compiled in the `PORT` macro (default 4242). An optional `-r <num_rooms>` flag sets the number of rooms (default 10, max 64). Game events are logged to `stdout`.

Client: The client is invoked as `./client <ip> <port>` and prompts the user for a player name before connecting. On connection, the client receives a room list and the user types `join <id>` or `new` to enter a room. Once in the lobby, they type `pick <id>` (0–7) to select a character. The host only needs to type `start` to begin. During each round, the user needs to type `<action_id> [target_id]` (e.g. `0`) to use their first move untargeted, or `2 3` to use their third move targeting player 3.

The client output has ANSI-coloured headers, a 20-character ASCII HP bar per player (rendered in `client.c`, `print_hp_bar`), and a typewriter-effect narrative display (15 ms delay between characters, `typewriter_print`) during resolution. Resolution refers to the time where the round ends and combat damage is calculated and messages are printed to the screen.

Key features

There are eight playable characters (Albert Guo, Kaitlyn Zhu, Yibin Wang, Doug Ford, TTC, The Bitter TA, Jesus Christ, Karen Reid) and each carry four unique moves, for a total of 32 distinct abilities. A type-effectiveness matrix (five types: Student, Politician, Infrastructure, Teaching Staff, Divine) provides 1.5× damage multipliers. A status-effect system tracks over 30 independent per-player fields (poison, scrambled moves, shields, invulnerability, evasion, healing blocks, permanent buffs/debuffs, etc.) all defined in the `StatusEffects` struct in `game.h`.

Each character has actions with different effects. Actions can be categorized into 3 types: attack, defend, affect self, or opponent. (code: char_def)

- Attack-type: lower the opponent's HP
- Defend-type: reduce or eliminate the damage caused to self
- Effect-type: create positive (e.g., healing) or negative (e.g., increase next round's damage) effects to self or opponent.

Type-effectiveness system (`game.c` — `get_type_effectiveness`): Characters have a "type". Attacks from a certain type of character deal 1.5× damage to specific opposing types — for example, a Teaching Staff-type attack deals 1.5× damage to a Student-type opponent. This feature is implemented via the `get_type_effectiveness` function in `game.c`, which encodes a fixed 5×5 advantage matrix and returns a multiplier of 1.5 when the attacker's type has an edge, or 1.0 otherwise.

Multi-room server (`server.c` — `rooms[]`): The server supports up to 64 simultaneous game rooms (10 by default, configurable with `-r`). Each room is an independent `GameState` and can hold up to 8 players and 16 spectators. Players are assigned to rooms explicitly via the room-selection screen, or auto-assigned to an empty room with the `new` command. All rooms run concurrently in the single-threaded event loop using `select`.

Status effects (`game.c` — `game_tick_effects`): Over 30 status effects are applied per-player each round by `game_tick_effects`. These include damage-over-time (poison), forced action skip (Drug Mixing), scrambled moves (Undefined Behaviour), forced miss (Dangling Pointer), evasion (Shuttle

Buses / Notwithstanding), shields (Office Hours), healing blocks (Fare Inspector), and permanent stat changes (Gradient Descent, Buck-a-Beer, Man Page Reading), among others.

Build Instructions

The project builds with a single `make` invocation. All source files must be present in the same directory. No external libraries beyond the standard C library and `libm` are required.

```
# Build both executables
make

# Optionally specify a different default port
make PORT=5000
```

This produces two executables: `server` and `client`.

Running the server:

```
./server
# Server listens on 0.0.0.0:4242 with 10 rooms

# Optional: specify a different port or room count:
./server -p 5000
./server -p 5000 -r 4
```

Running clients (one terminal per player):

```
./client 127.0.0.1 4242
```

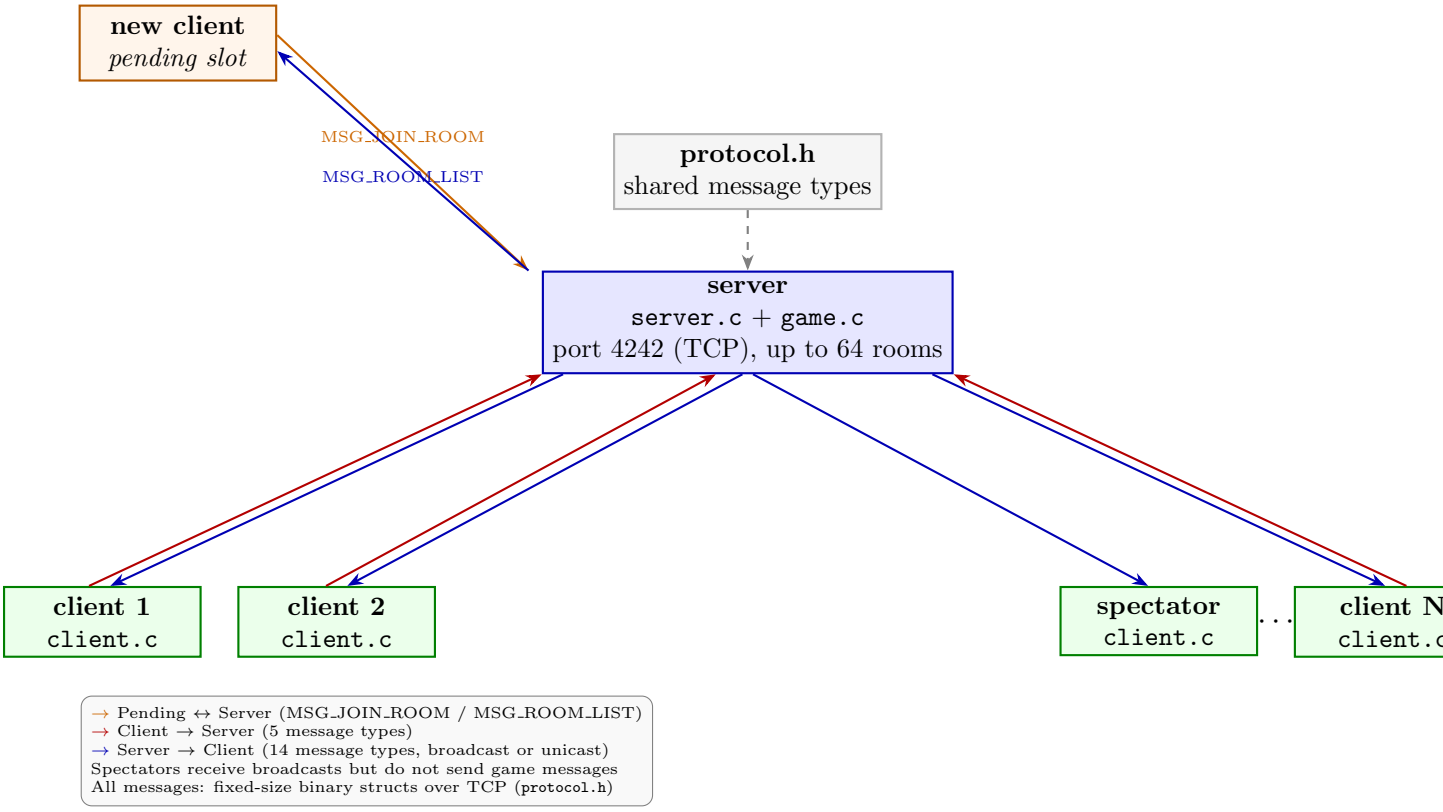
Minimum requirements: 2 players must connect to the same room and select characters before the host can type `start`. Up to 8 players and 16 spectators can be in a room simultaneously. The server continues running indefinitely; each room resets to the lobby after a game ends.

Compiler flags used (see `Makefile`):

```
gcc -Wall -Wextra -g -DPORT=4242 -o server server.c game.c -lm
gcc -Wall -Wextra -g -DPORT=4242 -o client client.c
```

Architecture Diagram

The diagram below shows the server process, up to eight client processes per room, the TCP socket connections between them, and the categories of messages that flow in each direction. All communication is defined in `protocol.h`.



The server's internal state machine (in `server.c` and `game.c`) is the sole source of truth. Clients maintain only the state required to render their UI and submit inputs. The `fd_set` maintained in `main` (server.c, lines 1231–1256) tracks the listen socket, all pending connection sockets, all spectator sockets, and every active player socket across all rooms simultaneously.

Communication Protocol and Error Handling

All messages are **fixed-width binary structs** defined in `protocol.h`. The first byte of every struct is always the `msg_type` opcode. Because TCP is a stream protocol, both sides must know how many bytes to read; the opcode alone determines the struct type and thus the exact byte count expected. The server dispatches incoming messages from players in `process_player_buffer` (server.c, line 967) and from pending connections in `process_pending_buffer` (server.c, line 673); the client dispatches server messages in `process_server_buffer` (client.c). The **Error Handling** column below documents what occurs when a message is malformed, arrives in the wrong game state, or when the underlying `write(2)` or `read(2)` call fails.

Client → Server messages

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
MSG_JOIN_ROOM (0x05)	36 bytes	`uint8_t msg_type; uint8_t room_id; uint8_t padding[2]; char name[32];`	Sent from a pending connection (before the client has joined any room). `room_id` is 0–63, or `AUTO_ROOM` (0xFF) to auto-assign to an empty lobby. `name` carries the player's chosen display name. If the target room is in `STATE_LOBBY`, the client becomes a player; if a game is in progress, they become a spectator.	**Server:** (1) Invalid `room_id` ⇒ `MSG_ERROR` + refreshed `MSG_ROOM_LIST` sent back (server.c:526–535). (2) Lobby full ⇒ `MSG_ERROR` + `MSG_ROOM_LIST` (server.c:542–551). (3) Too many spectators ⇒ `MSG_ERROR` + `MSG_ROOM_LIST` (server.c:614–624). (4) `AUTO_ROOM` with no empty room available ⇒ `MSG_ERROR` + `MSG_ROOM_LIST` (server.c:512–523). Non-printable bytes in `name` are sanitised to `'?'` (server.c:490–491); an empty name is replaced with "Player". **Client send:** `write_all` failure ⇒ `perror("write")` + `exit(1)` (client.c:514–516).
MSG_JOIN (0x01)	33 bytes	`uint8_t msg_type; char name[32];`	Legacy opcode handled by `handle_join` (server.c:794). Can be sent by already-slotted player fds. In the current client flow, players enter rooms via `MSG_JOIN_ROOM` instead; `MSG_JOIN` is retained for compatibility but is not sent by the updated client.	**Server:** (1) Game already in progress ⇒ `MSG_ERROR` + close socket (server.c:798–804). (2) Lobby full ⇒ `MSG_ERROR` + close socket (server.c:814–820). (3) Non-printable bytes in `name` sanitised to `'?'` (server.c:810–811).
MSG_ACTION				

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
(0x02)	4 bytes	`uint8_t msg_type; uint8_t action_id; uint8_t target_id; uint8_t padding;`	Sent during `STATE_ACTION_COLLECTION` only. `action_id` is 0–3. `target_id` is forced to `NO_TARGET` (0xFF) for untargeted moves. On success the server sends `MSG_WAIT`.	**Server:** (1) Wrong game phase ⇒ `MSG_ERROR` "Not currently accepting actions." (server.c:881–885). (2) Duplicate submission ⇒ `MSG_ERROR` "You already submitted an action this round." (server.c:886–890); original action preserved. (3) Invalid `action_id` or dead/out-of-range target ⇒ `MSG_ERROR` with specific string from `game_validate_action` (server.c:896–900); connection stays open so player can resubmit. **Client send:** `write_all` failure ⇒ `perror("write")` + `exit(1)` (client.c:580–583). Client also guards locally: a targeted move submitted without a target ID prints a usage hint without sending (client.c:567–572).

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
MSG_START_GAME				
(0x03)	4 bytes	`uint8_t msg_type; uint8_t padding[3];`	Sent by the host to begin the game. Only accepted during `STATE_LOBBY` with ≥2 connected players, all having selected characters.	**Server:** (1) Non-host sender ⇒ `MSG_ERROR` "Only the host can start." (server.c:926–930); connection left open. (2) Game already started ⇒ `MSG_ERROR` "Game already started." (server.c:931–935). (3) Any connected player has no character ⇒ `MSG_ERROR` "All players must select a character first." (server.c:941–945). (4) Fewer than 2 players ⇒ `MSG_ERROR` "Need at least 2 players to start." (server.c:950–954). In all four cases the game stays in `STATE_LOBBY` and no state is mutated. **Client send:** `write_all` failure ⇒ `perror("write") + exit(1)` (client.c:467–470).
MSG_CHAR_SELECT				
(0x04)	4 bytes	`uint8_t msg_type; uint8_t char_id; uint8_t padding[2];`	Sent during `STATE_LOBBY` to claim character slot 0–7. On success, `MSG_LOBBY_UPDATE` and updated `MSG_CHAR_LIST` are broadcast to all players (server.c:864–870).	**Server:** (1) Not in lobby phase ⇒ `MSG_ERROR` "Can't change character now." (server.c:848–851); connection left open. (2) `char_id` out of range or character already taken ⇒ `MSG_ERROR` "Character unavailable or invalid." (server.c:854–857); player may retry with a different ID. **Client send:** `write_all` failure ⇒ `perror("write") + exit(1)` (client.c:452–455).

Server → Client messages

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
MSG_ROOM_LIST				
(0x1C)	260 bytes	`uint8_t msg_type; uint8_t room_count; uint8_t padding[2]; RoomInfo rooms[64];`	Unicast to every new TCP connection immediately after `accept` (server.c:1350–1352). Also re-sent to a pending client when their `MSG_JOIN_ROOM` is rejected (invalid room, room full, etc.) so they can retry. Each `RoomInfo` (4 bytes) encodes room id, player count, phase, and spectator count.	**Server write:** queued via `queue_write_pending` (server.c:1352); flushed the next time the pending fd appears in `wfds`. If `queue_write_pending` would overflow the 8 KB pending write buffer, it returns `-1`; the accept loop does not check this return value, so in the extreme case the room list may be silently dropped. **Client receive:** triggers `on_room_list` (client.c:136), which sets `in_room_select = 1` and displays the room table; always structurally valid.
MSG_SPECTATE_START				
(0x1D)	324 bytes	`uint8_t msg_type; uint8_t player_count; uint8_t round_num; uint8_t padding; PlayerInfo players[8];`	Unicast to a client the moment they are placed in a spectator slot (server.c:665). Carries the current round number and a snapshot of all player HP, alive status, and character type so the spectator can immediately render the in-progress state.	**Server write:** sent via `queue_write_spectator` inside `send_spectate_start` (server.c:394–403); flushed on the next `wfds` iteration. **Client receive:** triggers `on_spectate_start` (client.c:158), which sets `is_spectator = 1`, `in_room_select = 0`, and renders the spectator view; always structurally valid.
MSG_JOIN_ACK				
(0x19)	4 bytes	`uint8_t msg_type; uint8_t your_id; uint8_t is_host; uint8_t padding;`	Unicast to the connecting client immediately after `MSG_JOIN_ROOM` is processed (lobby path) or after a spectator is promoted to lobby player by `promote_spectators`. `your_id` is the player's slot (0–7); `is_host` is 1 if this player is the game host.	**Server write:** queued via `queue_write` in `send_join_ack` (server.c:366–374); returns `-1` on failure. The caller checks the return value and calls `disconnect_player` if the send fails, aborting the join sequence before any lobby broadcast occurs. **Client receive:** triggers `on_join_ack` (client.c:182), which sets `in_room_select = 0` and `is_spectator = 0`; always structurally valid.
MSG_CHAR_LIST				

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
(0x1A)	≈13 KB	`uint8_t msg_type; uint8_t char_count; uint8_t padding[2]; CharEntry chars[8];`	Unicast on join and broadcast after every character selection. Each `CharEntry` (protocol.h) contains stats and all four `ActionDef` entries. The `taken` field marks already-claimed characters.	**Server write:** sent via `queue_write` in `send_char_list` (server.c:255–263); returns `-1` on failure. All callers (`handle_join_room`, `handle_char_select`, `promote_spectators`) check the return value and call `disconnect_player` on failure. **Client receive:** always structurally valid.
MSG_LOBBY_UPDATE				
(0x10)	≈324 bytes	`uint8_t msg_type; uint8_t player_count; uint8_t host_id; uint8_t padding; PlayerInfo players[8];`	Broadcast to all clients whenever the lobby changes (player joins, leaves, or picks a character). Each `PlayerInfo` (40 bytes) encodes player id, hp, alive flag, char type, speed, max hp, char id, and name.	**Server write:** sent via `broadcast` (server.c:199); `queue_write` failure for any slot ⇒ `disconnect_player` called immediately for that slot; remaining players continue to receive the update uninterrupted. **Client receive:** always structurally valid.

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
MSG_GAME_START				
(0x11)	≈1.4 KB	`uint8_t msg_type; uint8_t your_id; uint8_t player_count; uint8_t action_count; PlayerInfo players[8]; ActionDef actions[4];`	Unicast to each player (server.c:265–285). `actions` carries the four moves for *that player's* chosen character so the client can display them without hardcoding game data.	**Server write:** sent per-player via `queue_write` (server.c:282); the return value is checked for each player, and `disconnect_player` is called immediately on failure. **Client receive:** always structurally valid; stores `my_id` and the action list for the remainder of the session.
MSG_ROUND_START				
(0x12)	≈324 bytes	`uint8_t msg_type; uint8_t round_num; uint8_t timer_secs; uint8_t player_count; PlayerInfo players[8];`	Broadcast to all players *and spectators* at the start of every action-collection phase via `broadcast_all` (server.c:296). Contains a fresh snapshot of all player HP, speed, and alive status.	**Server write:** sent via `broadcast_all`; `queue_write` failure ⇒ `disconnect_player` or `disconnect_spectator`. **Client receive:** resets per-round state flags (`waiting_for_result`, `in_resolution`) and sets `need_prompt = 1` for players (spectators receive the update but suppress the action prompt); always structurally valid.
MSG_TIMER_TICK				
(0x13)	4 bytes	`uint8_t msg_type; uint8_t seconds_left; uint8_t padding[2];`	Broadcast to players and spectators once per second during action collection (server.c:1394–1396). When `seconds_left` reaches 0 the server proceeds to resolution regardless of missing submissions.	**Server write:** sent via `broadcast_all` (server.c:306); `queue_write` failure ⇒ `disconnect_player` or `disconnect_spectator`. **Client receive:** always structurally valid; updates the countdown display or a "waiting" status line depending on `waiting_for_result`.
MSG_ROUND_EVENT				
(0x14)	260 bytes	`uint8_t msg_type; uint8_t padding[3]; char text[256];`	Broadcast to players and spectators during resolution, one event every two seconds (`RESOLVE_DELAY_US` = 2 000 000 µs, server.c:25). Each event is a null-terminated narrative string describing one combat action.	**Server write:** sent via `broadcast_all` (server.c:315); `queue_write` failure ⇒ `disconnect_player` or `disconnect_spectator`. **Client receive:** always structurally valid; sets `in_resolution = 1`, which suppresses the action prompt until the next `MSG_ROUND_START` is received.

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
MSG_STATUS_UPDATE				
(0x1B)	516 bytes	`uint8_t msg_type; uint8_t padding[3]; char text[512];`	Unicast to each living player at the start of every round (server.c:1074–1082). Contains a human-readable summary of active status effects built by `game_build_status_text` (game.c).	**Server write:** sent via `queue_write` only to players with `alive && fd != -1` (server.c:1076–1081); the return value is checked and `disconnect_player` is called on failure. **Client receive:** always structurally valid.
MSG_PLAYER_ELIM				
(0x15)	36 bytes	`uint8_t msg_type; uint8_t player_id; uint8_t padding[2]; char name[32];`	Broadcast to players and spectators when a player's HP reaches zero or they disconnect mid-game. Clients display a "player eliminated" notification.	**Server write:** sent via `broadcast_all` (server.c:334); `queue_write` failure ⇒ `disconnect_player` or `disconnect_spectator`. **Client receive:** always structurally valid.
MSG_GAME_OVER				
(0x16)	36 bytes	`uint8_t msg_type; uint8_t is_tie; uint8_t winner_id; uint8_t padding; char winner_name[32];`	Broadcast to players and spectators when one (or zero) players remain. `is_tie` is 1 if multiple players eliminated simultaneously; `winner_id` is 0xFF on a tie. After broadcasting, the server resets the room to `STATE_LOBBY` and calls `promote_spectators`.	**Server write:** sent via `broadcast_all` (server.c:346); `queue_write` failure ⇒ `disconnect_player` or `disconnect_spectator`. After this message the server resets to lobby and spectators are promoted. **Client receive:** sets `game_over = 1` and displays the result; always structurally valid.

Message / Opcode	Size	Encoding	Semantics & Invariants	Error Handling
MSG_WAIT (0x18)	4 bytes	`uint8_t msg_type; uint8_t padding[3];`	Unicast acknowledgement sent to a player after their `MSG_ACTION` is accepted (server.c:913). Signals the client to stop prompting for input and wait for resolution.	**Server write:** sent via `queue_write` in `send_wait` (server.c:358–364); returns `-1` on failure. The caller in `handle_action` checks the return value and calls `disconnect_player` on failure. The player's `pending_action_id` is recorded *before* the send (server.c:910–911), so the action is preserved even if delivery fails. **Client receive:** sets `waiting_for_result = 1` and suppresses further input prompts; always structurally valid.
MSG_ERROR (0x17)	68 bytes	`uint8_t msg_type; uint8_t padding[3]; char message[64];`	Unicast error string for invalid actions, bad character selections, unauthorised start requests, rejected room joins, and connection-refused conditions. Fatal errors are followed immediately by socket close or a refreshed room list.	**Server write:** sent via `queue_write` in `send_error` (server.c:349–356); returns `-1` on failure. For non-fatal errors sent to registered players (e.g. invalid actions, bad character selections), the caller checks the return value and calls `disconnect_player` on failure. For rejected room-join attempts from pending connections, the server re-sends `MSG_ROOM_LIST` immediately after the error so the client can retry. **Client receive:** prints the message in red (`on_error`, client.c); connection stays open for non-fatal errors so the player may retry.

Message framing

Because TCP is a byte stream, partial writes are possible. Both sides use a `write_all` helper (server.c, line 69; client.c, line 47) that loops on `write(2)` until all bytes are sent or an error occurs. On the receive side the server maintains a 256-byte per-player read buffer (`Player.rbuf`, game.h) and a per-pending-connection read buffer (`PendingConn.rbuf`) that accumulate partial reads; `process_player_buffer` (server.c, line 967) and `process_pending_buffer` (server.c, line 673) each check whether a full message is present before dispatching, and call `memmove` to compact the buffer afterwards. When a pending connection is promoted to a player slot, only the bytes *following* the already-processed `MSG_JOIN_ROOM` are transferred into the player's rbuf, preventing stale opcode bytes from being re-dispatched (server.c:571–577). The client uses a single 8 KB `rbuf` (client.c,

line 28) for the same purpose.

Cross-cutting error handling

The following concerns affect the protocol as a whole rather than any individual message type.

1. Client disconnection mid-game. Every `read` on a player socket is checked for $n \leq 0$ (server.c, line 1441–1451). Zero means a clean peer close; negative means an I/O error (with an `EAGAIN/EWOULDBLOCK` exception for non-blocking sockets, which simply causes the read to be skipped). In both fatal cases `disconnect_player` is called (server.c, line 1445), which closes the socket, sets `fd = -1` to remove it from the `fd_set` on the next iteration, and frees the character slot. If the game is in progress, `game_remove_player` marks the player as eliminated and `MSG_PLAYER_ELIM` is broadcast to remaining players. `game_check_end` handles the edge case where a disconnect reduces survivors to one.

2. Unknown or malformed opcode. In `process_player_buffer` (server.c, line 980–984), the `default` branch logs the unknown opcode and immediately calls `disconnect_player`, preventing the server from interpreting arbitrary bytes as valid messages and corrupting game state. Similarly, in `process_pending_buffer` (server.c, line 682–688), an unknown opcode causes `disconnect_pending` to be called. Symmetrically, in `process_server_buffer` (client.c), an unknown server opcode causes a return of `-1`, which breaks the client event loop and closes the socket.

3. Write failures (broken pipe). All outbound data is routed through `queue_write` (server.c, line 39) and flushed by `flush_wbuf` (server.c, line 51). `flush_wbuf` returns `-1` on any fatal `write(2)` failure including `EPIPE`. Equivalent helpers exist for pending connections (`queue_write_pending`, `flush_wbuf_pending`) and spectators (`queue_write_spectator`, `flush_wbuf_spectator`). Broadcast helpers that use `broadcast_all` check the return value of `queue_write` per player and call `disconnect_player` or `disconnect_spectator` on failure. If `queue_write` detects that the 8 KB buffer would overflow, it also returns `-1`, disconnecting the client. This design means a slow or unresponsive client can never stall the server.

4. Socket setup failures. Every system call in the server startup sequence—`socket`, `setsockopt`, `bind`, and `listen`—checks its return value and calls `perror` followed by `exit(1)` on failure (server.c, lines 1188–1205). The client similarly checks `socket`, `inet_pton`, and `connect` (client.c, lines 612–627).

5. `select` interrupted by signal (EINTR). Both the server (server.c, line 1301) and the client (client.c, line 641) check `errno == EINTR` after a negative `select` return and restart the loop with a `continue`. Any other negative return is treated as fatal and reported with `perror`:

```
if (ready < 0) {
    if (errno == EINTR) continue;
    perror("select"); exit(1);
}
```

6. Client stdin EOF. In both the lobby handler (client.c, line 438–441), the room-select handler (client.c, line 500–503), and the in-game handler (client.c, line 540–543), a `NULL` return from `fgets` closes the socket and calls `exit(0)`. Closing the socket triggers `read` returning 0 on the server, which invokes `disconnect_player` or `disconnect_pending` and handles the departure identically to any other disconnection.

7. Player eliminated by status-effect tick before action collection. `begin_round` (server.c, line 1015) calls `game_tick_effects` and then immediately calls `game_check_end` before transitioning to `STATE_ACTION_COLLECTION`. If the check returns non-zero (one or zero survivors), the server broadcasts `MSG_PLAYER_ELIM` for each newly dead player, resets to `STATE_LOBBY`, promotes spectators, and sends `MSG_GAME_OVER` without opening the action-collection window, ensuring the game always terminates cleanly even when damage-over-time effects deliver the killing blow between rounds.

8. SIGPIPE suppression. Writing to a TCP socket whose peer has closed produces a `SIGPIPE` signal, which by default terminates the process. The server installs `signal(SIGPIPE, SIG_IGN)` at the top of `main` (server.c, line 1166) so that `write(2)` returns `-1` with `errno == EPIPE` instead. The existing `flush_wbuf` and `queue_write` error paths already check for negative return values and route through `disconnect_player`, so once `SIGPIPE` is suppressed the broken-pipe case is handled identically to any other write failure—no special-case code is needed.

Concurrency Model

The server is **single-process, single-threaded**. It uses a single `select(2)` call to monitor all file descriptors simultaneously, never blocking on any one client. No `fork` or `pthread` calls are made; concurrency is achieved entirely through I/O multiplexing.

The `fd_set` contents

At the top of the main event loop (server.c, lines 1224–1256) the server rebuilds `rfds` and `wfds` on every iteration:

- **The listen socket** (`listen_fd`) — added to `rfds` to monitor for new incoming TCP connections.
- **All pending connection sockets** (up to 64) — added to `rfds` for incoming `MSG_JOIN_ROOM` data, and to `wfds` if their write buffer is non-empty (`wbuf_len > 0`) so that `MSG_ROOM_LIST` or error messages can be flushed asynchronously (server.c:1249–1256).
- **All active player sockets** (up to 8 per room, across all rooms) — added to `rfds` for incoming data. Any player socket with a non-empty write buffer is also added to `wfds` so the server can flush pending outbound data as soon as the kernel is ready to accept it (server.c:1231–1239).
- **All spectator sockets** (up to 16 per room, across all rooms) — added to `rfds` for disconnection detection, and to `wfds` if their write buffer is non-empty so that game-event messages forwarded to spectators are flushed asynchronously (server.c:1240–1248).

Timeout logic

`select` is called with a baseline `struct timeval` of 100 ms (server.c, lines 1260–1263). Before calling `select`, the server iterates over all rooms and refines the timeout downward (server.c, lines 1265–1297):

- `STATE_ACTION_COLLECTION`: room requests a 1-second timeout. The server decrements `timer_secs_left` each second, broadcasts `MSG_TIMER_TICK`, and if the timer reaches zero calls `start_resolution` immediately.
- `STATE_RESOLVING`: room requests a timeout equal to the time remaining until the next 2-second event slot, computed using `gettimeofday`. When the timeout fires the server sends the next queued narrative event (`MSG_ROUND_EVENT`) and updates `resolve_last_send`.
- `STATE_LOBBY` / `STATE_GAME_OVER`: no reduction; the 100 ms baseline is used (effectively polling, allowing new connections and messages to be processed promptly even when all rooms are idle).

The minimum timeout across all rooms wins, so a single `select` call services every active timer simultaneously.

Non-blocking I/O and the write buffer

Every accepted socket (players, pending connections, spectators) is set to `O_NONBLOCK` via `set_nonblocking` (server.c, line 31) immediately after `accept`. This guarantees that neither `read` nor `write` can block the server on a single slow client.

On the **read side**, every `read` is gated behind `FD_ISSET`, so the server never attempts a read on a socket that has no data. For non-blocking player sockets, `EAGAIN/EWOULDBLOCK` is explicitly handled

as a non-fatal condition (server.c:1444), which is important when a pending connection is promoted to a player slot within the same `select` iteration and the new player fd appears ready but has no new data.

On the **write side**, the server never calls `write(2)` directly on a client socket during message dispatch. Instead, all send helpers call `queue_write` (or `queue_write_pending` / `queue_write_spectator`), which appends the message to the client's 8 KB write buffer. After `select` returns, any socket that appears in `wfds` is flushed by the appropriate `flush_wbuf` variant, which handles `EAGAIN/EWOULDBLOCK` by returning immediately—the remaining data stays in the buffer and is retried on the next `select` iteration. This design means a slow or unresponsive client can never stall the server; data accumulates in the per-client buffer and is drained opportunistically.

Early exit when all actions are received

After processing each player's message, the server checks `game_all_actions_in` (server.c, line 1472). If every living player has submitted, resolution begins immediately without waiting for the timer to expire, keeping rounds snappy in practice.

Client event loop

The client also uses `select` (client.c, line 640) to multiplex between two readable sources: the server socket and `stdin`. This lets the client simultaneously watch for incoming server messages (timer ticks, round events) while accepting the player's move input without blocking on either. The client's stdin routing is controlled by the `in_room_select` and `is_spectator` flags: pending room selection dispatches to `handle_stdin_room_select`; spectators drain stdin silently; active players dispatch to `handle_stdin_game` or `handle_stdin_lobby` depending on whether the game has started.

Team Contributions

Component	Albert	Kaitlyn	Yibin
`protocol.h` — message struct definitions	•		
`game.h` — data structure design	•		•
`game.c` — character & move definitions			•
`game.c` — damage pipeline (`apply_damage`)	•		
`game.c` — status effect system	•		•
`game.c` — action resolution & sorting	•		
`game.c` — type effectiveness matrix			•
`server.c` — TCP setup & accept loop	•		
`server.c` — `select` event loop & timer logic	•		
`server.c` — message handlers	•		
`server.c` — resolution drip-feed logic	•		
`server.c` — disconnect & error handling	•	•	
`server.c` — multi-room & room selection	•		
`server.c` — spectator mode & promotion	•		
`client.c` — socket connection & send logic	•		
`client.c` — message receive & dispatch	•		
`client.c` — room selection UI	•		
`client.c` — terminal UI (HP bars, ANSI)	•		
`client.c` — typewriter effect & timer display	•		
`Makefile` — build system	•		
Project documentation (this report)		•	•
Demo video		•	

Albert Guo designed and implemented the core client–server architecture, including the TCP socket setup, the `select`-based event loop, the binary communication protocol (`protocol.h`), and all message handlers on both the server and client sides. He also built the game engine's damage pipeline, action resolution and sorting logic, the status effect system, and the full client terminal UI (HP bars, ANSI colouring, typewriter effect, timer display). He wrote the `Makefile` and contributed to the server-side disconnect and error handling.

Kaitlyn Zhu focused on error handling and robustness across the codebase, including the server-side disconnect logic, write-failure recovery in `broadcast` and unicast helpers, and validation of edge cases such as unknown opcodes and mid-game disconnections. She was the primary author of this project report and produced and edited the demo video.

Yibin Wang designed and implemented the game content: the eight playable characters, their four unique moves each (32 abilities total), the five-type effectiveness matrix (`get_type_effectiveness` in `game.c`), and the character class system. She built the multi-room server architecture supporting up to 64 independent game rooms, the room-selection screen, and the spectator system (including `promote_spectators` which automatically transitions spectators into lobby players after a game ends). She also contributed to the data structure design in `game.h` and the status effect system in `game.c`, and helped write sections of this report.